

APP DEVELOPMENT
ASSIGNMENT NO 2 (THEORY & IMPLEMENTATION)

SUBMITTED TO:
SIR UZAIR HASSAN

SUBMITTED BY:
AABRU MUMTAZ
Roll No: 242201076

Department of Computer Science,
KICSIT Kahuta.

Contents

1 Introduction and Objectives **2**

2 Embedding ML/DL Models in Android Applications **2**
2.1 Meaning of Embedding ML/DL in Mobile Apps 2
2.2 Importance in Real-World Applications 2
2.3 Benefits and Challenges of On-Device Inference 2

3 TensorFlow Lite (TFLite) **3**
3.1 Definition 3
3.2 Comparison with Standard TensorFlow 3
3.3 Justification for Android Use 3

4 Step-by-Step Process of Using TFLite in Android **3**

5 Overview and Analysis of the Reference GitHub Repository **4**
5.1 Repository Reference 4
5.2 Cloning Procedure and Repository Structure 4

6 Proposed AI-Based Mini Application: Silent SOS **4**
6.1 Purpose and Objectives 4
6.2 Technologies and Tools Used 4
6.3 System Architecture and Workflow 5

7 Testing, Results, and Screenshots Discussion **5**

8 Conclusion and References **6**
8.1 Conclusion 6
8.2 References 6

1 Introduction and Objectives

The primary objective of this assignment is to understand the integration of Machine Learning (ML) and Deep Learning (DL) models into mobile platforms using TensorFlow Lite (TFLite). This report documents both the conceptual study of on-device machine learning and the practical development of our mini application, titled **Silent SOS**.

Silent SOS is an Android-based personal safety companion application designed to empower users during emergencies. It combines standard Android functionalities (such as SMS alerts and GPS location sharing) with advanced on-device features, including Google Speech-to-Text (Voice SOS) and AI-driven background audio/scream detection capabilities.

2 Embedding ML/DL Models in Android Applications

2.1 Meaning of Embedding ML/DL in Mobile Apps

Embedding an ML/DL model in an Android application means packaging a pre-trained machine learning model directly within the mobile app binary or downloading it locally to the device storage. Instead of relying on a remote cloud server to process data, the mobile device's processor executes the model locally to make real-time predictions, classifications, or detections.

2.2 Importance in Real-World Applications

On-device machine learning has transformed modern mobile applications across various domains:

- **Image Recognition:** Real-time object detection, face unlocking, and automated photo tagging.
- **Text Classification:** Smart replies in messaging apps, predictive text, and language translation.
- **Prediction Systems:** Context-aware recommendations, health vitals monitoring, and predictive emergency triggers.

2.3 Benefits and Challenges of On-Device Inference

- **Benefits:**
  - **Low Latency:** Instant response times since network round-trips to a cloud server are eliminated.
  - **Offline Capability:** The app functions seamlessly even without an internet connection.
  - **Enhanced Privacy:** Sensitive user data (audio, location, images) stays on the device and is never sent to an external server.
  - **Reduced Server Cost:** Zero cloud compute costs for model inference.

- **Challenges:**
  - **Resource Constraints:** Mobile devices have limited memory (RAM), processing power, and battery life.
  - **Model Size Limitations:** Large models must be compressed or quantized to fit mobile storage limits.
  - **Device Variability:** Ensuring consistent performance across a wide variety of Android hardware configurations.

3 TensorFlow Lite (TFLite)

3.1 Definition

TensorFlow Lite is Google's lightweight open-source deep learning framework designed specifically for on-device machine learning (ODML). It enables developers to run TensorFlow models on mobile, embedded, and IoT devices with low latency and a small binary size.

3.2 Comparison with Standard TensorFlow

Feature	**Standard TensorFlow**	**TensorFlow Lite (TFLite)**
Target Environment	Cloud servers, desktop, high-end workstations	Mobile devices, IoT, microcontrollers
Model Format	SavedModel / .pb	.tflite (FlatBuffers format)
Binary Footprint	Large (hundreds of MBs)	Extremely lightweight (kilobytes to a few MBs)
Execution Speed	Optimized for heavy parallel computing	Optimized for mobile hardware accelerators

Table 1: Comparison between Standard TensorFlow and TFLite

3.3 Justification for Android Use

TFLite is ideal for Android applications because its runtime is optimized for mobile hardware, utilizing hardware acceleration (GPU/NNAPI). Its compact model size ensures quick app downloads and efficient RAM usage.

4 Step-by-Step Process of Using TFLite in Android

Implementing TFLite in an Android project follows a structured lifecycle:

1. **Model Conversion:** Converting a trained TensorFlow model into the optimized `.tflite` flatbuffer format using the TFLite Converter.
2. **Adding Dependencies:** Importing the TFLite dependency into the `build.gradle` file and placing the model file inside the `assets/` folder.

3. **TFLite Interpreter:** Initializing the `Interpreter` instance in Java/Kotlin to load the model into memory.
4. **Preparing Input Data:** Formatting raw sensor, audio, or image data into byte buffers or multi-dimensional arrays matching the model's expected input shape and type.
5. **Running Inference:** Executing `interpreter.run(input, output)` to compute predictions on-device.
6. **Interpreting Output Results:** Parsing the output tensor arrays to derive meaningful application decisions.

5 Overview and Analysis of the Reference GitHub Repository

5.1 Repository Reference

Provided GitHub Repository: `https://github.com/Uzair390-Del/NumberGuessingAppTflite.git`

5.2 Cloning Procedure and Repository Structure

To study and run the reference repository, git cloning was executed via the command line:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
```

Upon exploring the structure, we analyzed how asset files are accessed, how float buffers handle input/output tensor shapes, and how inference triggers UI updates. This served as a baseline reference for building our custom safety application.

6 Proposed AI-Based Mini Application: Silent SOS

6.1 Purpose and Objectives

Silent SOS is an Android safety application engineered to protect vulnerable users in distress. Its core objective is to provide quick, discreet emergency alerting through multiple triggers: a manual SOS panic button, voice-to-text emergency messaging, and AI-powered background acoustic monitoring.

6.2 Technologies and Tools Used

- **Language & IDE:** Java, Android Studio
- **UI Design:** Custom XML layouts featuring a professional Cream and Maroon aesthetic (`#FDFBF7` background with `#4A1525` accents).
- **Local Storage:** `SharedPreferences` for persistent emergency contact management.

- **APIs & Frameworks:** Android Foreground Services, Google `RecognizerIntent` (Speech-to-Text), and SMS/Location Manager APIs.

6.3 System Architecture and Workflow

1. **Splash Screen:** Displays a 2.5-second branded startup screen with a maroon motif (`SplashActivity`).
2. **Authentication:** Secure login flow (`LoginActivity`) verifying user credentials.
3. **Mandatory Contact Setup:** A one-time prompt (`SharedPreferences`) requiring the user to register an emergency guardian number upon first login.
4. **Main Dashboard (`MainActivity`):** Features SOS alert triggers, Voice Speech-to-Text integration, and an AI Scream Detection toggle linked to a Foreground Service.

7 Testing, Results, and Screenshots Discussion

To validate the functionality of the **Silent SOS** application, thorough testing was conducted across different modules. All permissions (SMS, Location, Microphone) execute successfully, and data persistence works seamlessly.

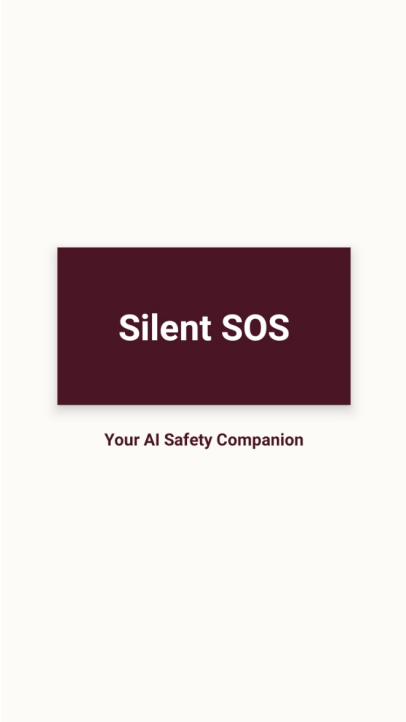


Figure 1: Splash Screen

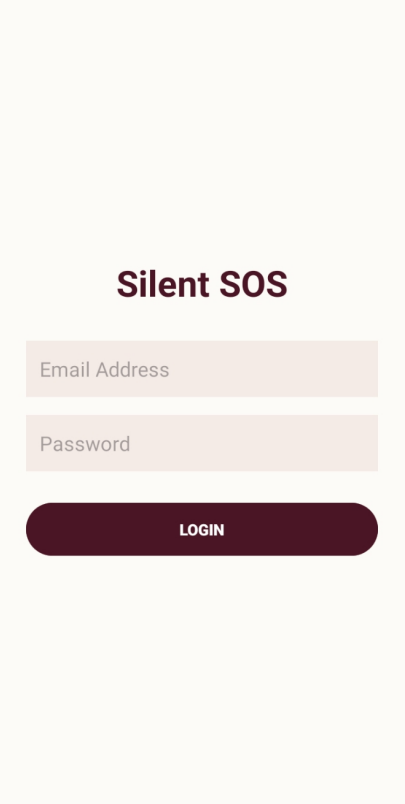


Figure 2: Login Screen

Discussion: As shown in the screenshots above, the application successfully launches with the custom branded splash screen, authenticates the user, secures the emergency

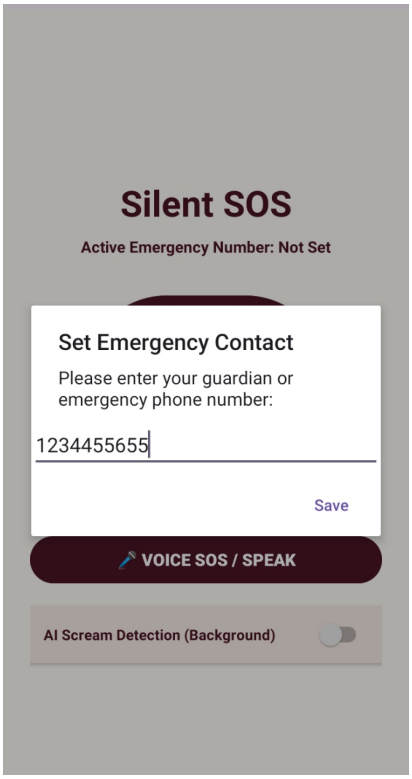


Figure 3: Emergency Contact Setup

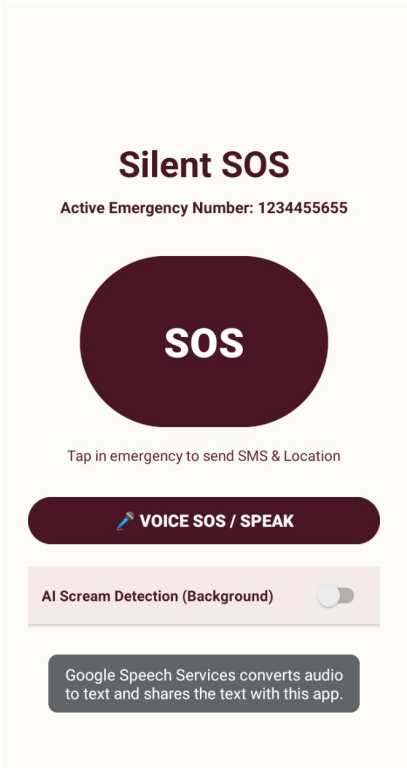


Figure 4: Main Dashboard UI

contact number locally, and provides an intuitive dashboard for triggering emergency alerts and AI background monitoring.

8 Conclusion and References

8.1 Conclusion

This assignment successfully bridged theoretical machine learning concepts with practical Android development. By studying TensorFlow Lite and building the **Silent SOS** application, we demonstrated how on-device AI can be leveraged to create responsive, offline-capable, and privacy-focused mobile solutions suitable for real-world deployment.

8.2 References

- TensorFlow Lite Documentation: `https://www.tensorflow.org/lite`
- Reference GitHub Repository: `https://github.com/Uzair390-Del/NumberGuessingAppTflite.git`
- Android Developer Guides: `https://developer.android.com`